OPENGL, GLUT, ZÁKLADY VYKRESLOVÁNÍ

Kurz: **Moderní počítačová grafika**

Lektor: Ing. Michal Švento

Úvod

- **OpenGL** je standardní specifikace, která definuje API (Application Programming Interface) pro tvorbu 2D a 3D grafiky, nezávisle na operačním systému, hardwaru a programovacím jazyku.
  - Funkce s prefixem `gl`
- **OpenGL Utility Library (GLU)** implementuje pomocné funkce, využívá funkcí OpenGL.
  - Funkce s prefixem `glu`
- **OpenGL Utility Toolkit (GLUT)** je knihovna, která zjednodušuje vývoj OpenGL aplikací poskytnutím jednoduchých příkazů, které zprostředkovávají funkce závislé na operačním systému (např. otevírání oken, interakci s uživatelem).
  - Funkce s prefixem `glut`
- Distribuce pro Visual Studio: NuGet balíček nupengl.core

Jak zajistíme vykreslení scény?

Schéma programu

Obslužné funkce

```
void onResize(int w, int h)
void onDisplay()
```

```
void vykresliDomecek()
void vykresliKruh(int numVertices, int X, int Y, int R)
void ukol1()
void ukol2()
```

Vykreslovací a jiné funkce

```
int main()

    inicializace GLUT

    vytvoření okna

    glutReshapeFunc(onResize)
    glutDisplayFunc(onDisplay)

    glutMainLoop()
```

GLUT funkce pro registraci událostí

Práce s GLUT

- **Inicializace** – nastavení stavových parametrů, které požadujeme před zahájením, otevření oken, registrace obsluh událostí.
  - `glutInit(int* argc, char** argv)`
  - `glutInitWindowSize(int width, int height)`
  - `glutInitWindowPosition(int x, int y)`
  - `int window = glutCreateWindow(char* title)`
  - `glutSetWindow(int window)`
  - `int window = glutGetWindow()`
- **Průběh** – spuštění nekonečné smyčky, ve které se již jen volají obsluhy událostí. Smyčku nelze přerušit, GLUT rozhoduje o jejím ukončení, po kterém by měl následovat konec aplikace.
  - `glutMainLoop()`

Definice událostí

- Obsluha události je funkce, jejíž parametry a návratová hodnota odpovídají předloze, která je dána typem události.
- Požadavek na překreslení, změna velikosti okna, interakce s uživatelem…
- Registrace funkce pomocí glut funkcí
  - `glutDisplayFunc(onDisplay)`
  - `glutReshapeFunc(onResize)`
  - `glutKeyboardFunc()`

Změna rozměrů okna

- Nastavení viewportů – specifikuje, jak budou převedeny normalizované souřadnice na souřadnice zobrazovacího zařízení:
  - `glViewPort(int x, y, width, height)`
- Nastavení aktuální transformační matice, na kterou se budou vztahovat všechny další operace:
  - `glMatrixMode()`
- Nastavuje aktuální matici na jednotkovou – vynuluje všechny transformace:
  - `glLoadIdentity()`
- Nastavení projekčních parametrů:
  - `glOrtho(double left, right, bottom, top, near, far)`

Překreslení okna

- Událost `onDisplay` je vyvolána pokaždé, když je třeba překreslit okno.
- **Nastává, je-li okno překryto, přesunuto, maximalizováno nebo je-li změněna jeho velikost.**

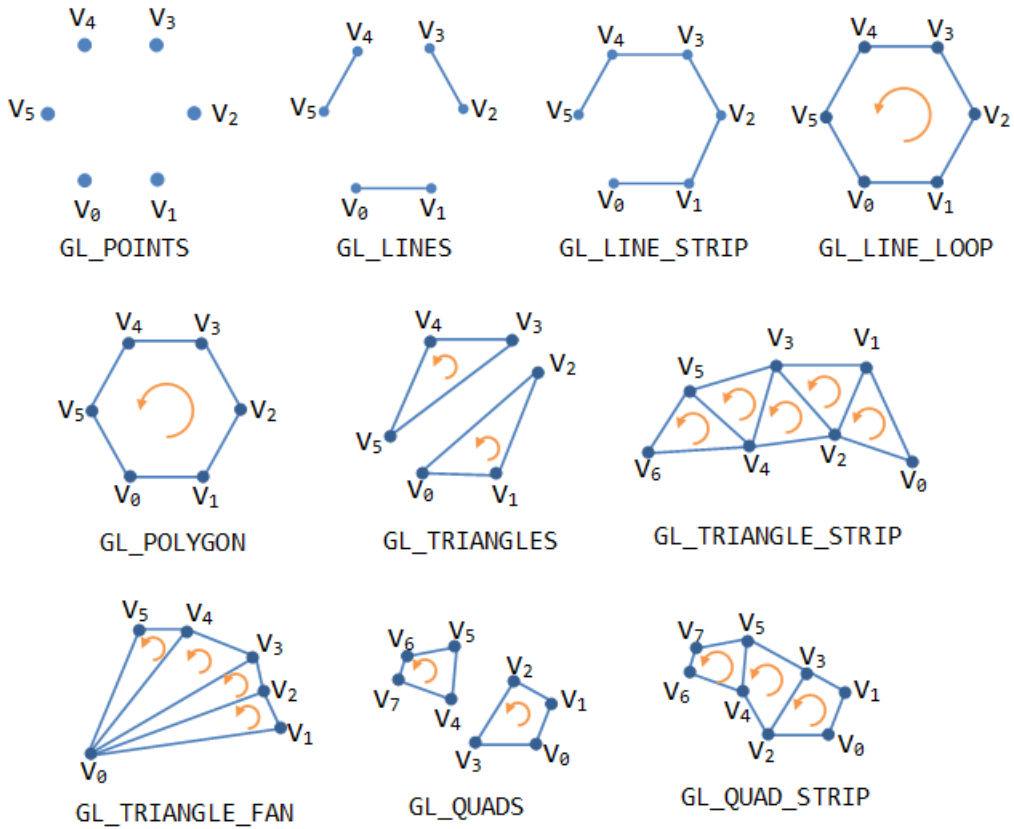
- Nastavení barvy mazání bufferu:
  - `glClearColor(float R, G, B, A)`
- Vyvolání mazání bufferu:
  - `glClear(GLbitfield mask)`
  - `GL_COLOR_BUFFER_BIT, GL_DEPTH_BUFFER_BIT, GL_ACCUM_BUFFER_BIT, GL_STENCIL_BUFFER_BIT`
  - Fyzické vykreslení definované scény na obrazovku:
  - `glFlush()`

Jak vytvoříme scénu

Vertexy a primitivy

- Vertex označuje abstraktní bod, který leží v prostoru OpenGL kontextu na definovaných souřadnicích.
- Definované vlastnosti vertexů:
  - barva
  - normálový vektor
  - optické vlastnosti materiálu
  - texturovací souřadnice
  - vlastnosti hrany
- Primitiva definuje, jakým způsobem má být vertex zobrazen.
- Kombinací vertexů získáme požadovanou primitivu.
- **Vertex je definován některou z následujících možností:**
  - `glVertex2i(int x, y)` `glVertex2f(float x, y)`
  - `glVertex3i(int x, y, z)` `glVertex3f(float x, y, z)`
  - `glVertex2iv(int*)` `glVertex2fv(float*)`
  - `glVertex3iv(int*)` `glVertex3fv(float*)`

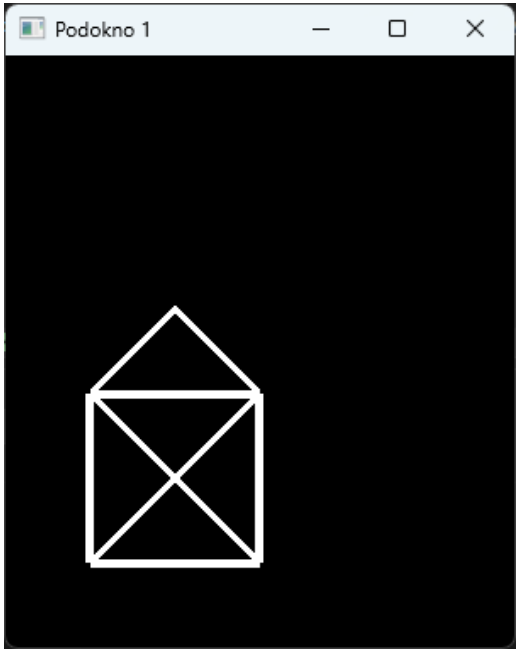
Primitivy



ukradeno z internetu

Úkol 1

- Projděte si tělo funkce `vykresliDomecek()` a doplňte kód tak, aby v okně 1 byl zobrazen domeček kreslený „jedním tahem“.



Úkol 2

- Parametrické vyjádření kružnice
  - $x(t) = s_x + r \cdot \cos(2\pi t)$, $y(t) = s_y + R \cdot \sin(2\pi t)$, $z = 0$,
  - $0 \leq t \leq 1$.

- Projděte hotový zdrojový kód a pochopte princip vykreslení kružnice. Sami podobně vykreslete parabolu s parametrickou rovnicí
  - $x(t) = 150 + 150t$, $y(t) = 300t^2$, $z = 0$,
  - $-1 \leq t \leq 1$.

Parametry vykreslování

- Nastavení barev
  - `glColor3f(float R, G, B)`
  - `glColor4f(float R, G, B, A)`
- Kreslení čar a bodů
  - Nastavení zobrazovacích bodů `glPointSize(float size)`
  - Nastavení šířky čáry `glLineWidth(float width)`
  - Definice patternu čáry `glLineStipple(int factor, short pattern)`

```
glEnable( { GL_POINT_SMOOTH
            GL_LINE_SMOOTH
            GL_LINE_STIPPLE
            GL_BLEND } )
```

```
glDisable( { GL_POINT_SMOOTH
             GL_LINE_SMOOTH
             GL_LINE_STIPPLE
             GL_BLEND } )
```

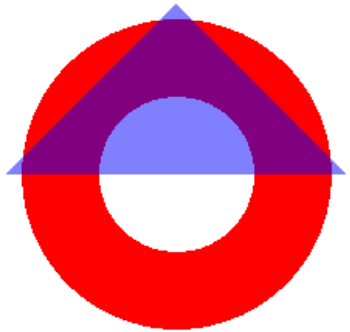
Kombinace barev

- Navázání objektu na pozadí nám zajistí práce s průhledností
  - `glEnable(GL_BLEND)` – povolení míchání barev
- `glBlendFunc(Glenum S, Glenum D)`
  - Rovnice míchání: $C_f = (C_s \cdot S) + (C_d \cdot D)$
  - C_f – výsledná barva
  - C_s – barva zdrojového objektu
  - C_d – barva cílového objektu
  - S – faktor průhlednosti zdrojového objektu
  - D – faktor průhlednosti cílového objektu

Parametr	RGB faktor průhlednosti
GL_SRC_COLOR	(Rs, Gs, Bs)
GL_ONE_MINUS_SRC_COLOR	(1, 1, 1) - (Rs, Gs, Bs)
GL_DST_COLOR	(Rd, Gd, Bd)
GL_ONE_MINUS_DST_COLOR	(1, 1, 1) - (Rd, Gd, Bd)
GL_SRC_ALPHA	(As, As, As)
GL_ONE_MINUS_SRC_ALPHA	(1, 1, 1) - (As, As, As)
GL_ZERO	(0, 0, 0)
GL_ONE	(1, 1, 1)

Úkol 3

- Pomocí míchání barev vytvořte následující obrázek:



  - Středy kružnic $(s_x, s_y) = (150, 150)$, poloměry $r_1 = 100$, $r_2 = 50$
  - Průhlednost $\alpha_\mathrm{R} = 30\%, \alpha_\mathrm{W} = 60\%, \alpha_\mathrm{B} = 80\%$
  - Souřadnice vertexů trojúhelníku

    $(s_x - r_1 - 10,\ \ s_y)$

    $(s_x + r_1 + 10,\ \ s_y)$

    $(s_x,\ \ s_y + r_1 + 10)$
  - Blending funkce `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`
  - Kód doplňte do funkce `vykresliPruhlednost()`

Antialiasing

- Při rasterizaci úsečky dochází k vytvoření skokových artefaktů.
- Odstranění pomocí kombinace **míchání barev a antialiasingových algoritmů.**

- `glEnable()`
  - `GL_POINT_SMOOTH, GL_LINE_SMOOTH, GL_POLYGON_SMOOTH`

    a zároveň `GL_BLEND`
- `glHint()`
  - Výběr algoritmu `GL_FASTEST, GL_NICEST`

- Multisampling
  - Vícenásobné vzorkování primitiv – vzorky jsou ukládány do speciálních pamětí.
  - `glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH | MULTISAMPLE)`
  - `glEnable(GL_MULTISAMPLE)`